

RESUMEN ARQUITECTURA DE SISTEMAS

CLIENTE/SERVIDOR Y MULTICAPAS

Arquitectura de Sistemas Cliente/Servidor	2
Definiciones y Conceptos Fundamentales	2
Características de la Operación	2
Clasificación de Gartner: Los 5 Modelos de Distribución	3
El Middleware: Tipos y Funciones (RPC, MOM, ORB)	3
Modelos de Distribución	3
Arquitecturas Multicapas	4
Modelo de 2 Capas	4
Modelo de 3 Capas	4
Ventajas e Inconvenientes	5
Arquitecturas de Servicios Web (WS) y Protocolos Asociados	5
SOA (Service Oriented Architecture)	5
Estándar SOAP (Simple Object Access Protocol)	6
Estándar WSDL (Web Services Description Language)	6
Otros Estándares y Protocolos	6
Formatos de Datos: XML (SOAP) vs. JSON (REST)	7
Arquitectura REST (Representational State Transfer)	7
Definición y Principios	7
Propiedades de los Métodos HTTP: Seguridad e Idempotencia	8
Métodos HTTP en REST	8
Arquitecturas Modernas: Microservicios y Contenedores	8
Microservicios	8
Kubernetes y Contenedores	9
Modelos de Servicio Cloud: IaaS, PaaS y SaaS	9

Arquitectura de Sistemas Cliente/Servidor

Definiciones y Conceptos Fundamentales

- **Sistemas Centralizados:** "Son aquellos en los que todos los componentes de una aplicación residen en un mismo equipo. Estos sistemas están limitados en escalabilidad puesto que su rendimiento es dependiente de la potencia de una única máquina. Habitualmente son sistemas monolíticos en los cuales no se puede modificar un elemento sin afectar al resto".
- **Arquitectura Cliente/Servidor:** "Modelo de distribución de aplicaciones donde las tareas se reparten entre los proveedores de recursos o servicios, llamados servidores, y los demandantes, llamados clientes".
- **Cliente:** "Se le conoce también con el término front-end. Se corresponde habitualmente con el navegador web del usuario".
- **Servidor:** "Procesador que proporciona servicios a los clientes".
- **Middleware:** "Capa de software intermedia que permite la comunicación entre el cliente y el servidor en sistemas heterogéneos".
- **DCE (Distributed Computing Environment):** "Constituye un estándar de facto para las tareas de comunicación en sistemas distribuidos, independientemente de su fabricante. Incluye servicios de RPC (Remote Procedure Call), seguridad, directorio, gestión de transacciones y archivo distribuido".

Características de la Operación

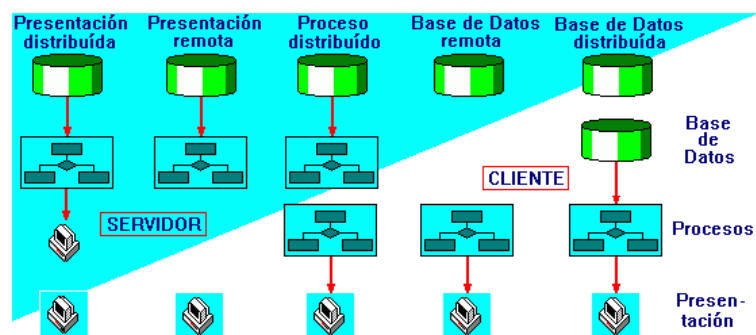
- **Interoperabilidad:** "Capacidad de sistemas diversos para trabajar juntos utilizando protocolos abiertos".
- **Concurrencia y paralelismo:** "Los servidores deben implementar los mecanismos de concurrencia y ejecución paralela necesarios para atender de forma concurrente a peticiones de distintos clientes".
- **Escalabilidad:** "Respuesta a la evolución de las necesidades de un sistema. Puede ser horizontal, si aumentamos o modificamos el número de equipos cliente, o vertical, si hacemos referencia a la posibilidad de aumento de la capacidad de cálculo de los servidores".

- **Acoplamiento:** "No es una característica propia de las arquitecturas cliente/servidor (el acoplamiento de sistemas es un distractor en examen)".

Clasificación de Gartner: Los 5 Modelos de Distribución

Este es el "santo grial" de las preguntas sobre Cliente/Servidor en las oposiciones de informática. Gartner clasifica los sistemas según dónde reside la Presentación, la Lógica y los Datos:

- **Presentación Distribuida:** El servidor tiene todo, el cliente solo actúa como terminal.
- **Presentación Remota:** La interfaz está en el cliente, el resto en el servidor.
- **Lógica Distribuida:** La lógica de negocio se reparte entre ambos.
- **Gestión de Datos Remota:** El cliente tiene presentación y lógica; el servidor solo la BD.
- **Bases de Datos Distribuidas:** El cliente tiene todo y la BD se reparte.



El Middleware: Tipos y Funciones (RPC, MOM, ORB)

- **Definición:** Software que asiste a una aplicación para que interactúe con otras (el "pegamento").
- **Tipos:** RPC (Remote Procedure Call), MOM (Message Oriented Middleware), ORB (Object Request Broker).

Modelos de Distribución

- **Thin Client (Cliente ligero):** "El servidor se encarga de todas las funciones excepto de la presentación".

- **Fat Client (Cliente pesado):** "El cliente realiza la mayor parte del procesamiento, limitando al servidor a funciones de datos".
- **Presentación remota:** "Los datos de que consta la base de datos se distribuyen entre múltiples clientes y servidores. La lógica primaria de la base de datos y de la aplicación permanecen en el servidor".

Enfoque CPS: El tribunal suele preguntar por la distinción entre escalabilidad vertical (más potencia al servidor) y horizontal (más servidores/clientes). Es un "clásico" preguntar qué elemento se conoce como front-end (cliente) y la característica del cliente ligero (solo hace la presentación).

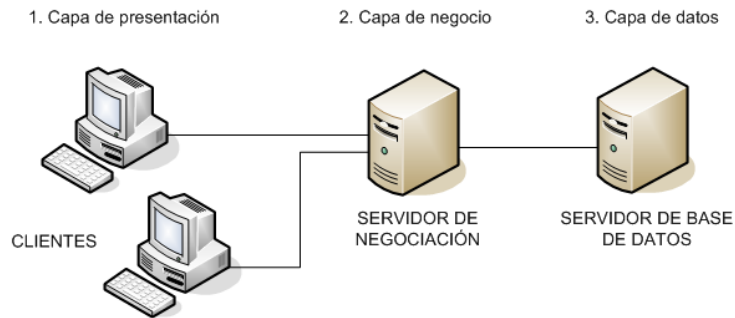
Arquitecturas Multicapas

Modelo de 2 Capas

- **"Arquitectura clásica donde el cliente se comunica directamente con el servidor de base de datos".**

Modelo de 3 Capas

- "Las aplicaciones se separarán en 3 capas, estableciendo una capa de Lógica de Presentación, una capa de Lógica de Negocio y una capa de Datos o de Persistencia".
- **Capa de Presentación:** "Capa que ve el usuario (GUI). Su función es capturar la información del usuario y mostrar los resultados".
- **Capa de Negocio (Lógica del Negocio):** "Aquí es donde se establecen todas las reglas que deben cumplirse. Se comunica con la de presentación para recibir solicitudes y con la de datos para pedir al gestor de base de datos almacenar o recuperar datos".
- **Capa de Datos:** "Donde residen los datos y es la encargada de acceder a los mismos. Formada por uno o más gestores de bases de datos (SGBD)".



Ventajas e Inconvenientes

- **Ventajas:** "Mayor flexibilidad, modularidad, escalabilidad y facilidad para añadir nuevas funcionalidades (extensibilidad) sin afectar a los clientes existentes".
- **Inconveniente (Trampa examen):** "Reducir el tráfico en la red NO es una ventaja propia de una arquitectura en 3 capas frente a una de dos capas".

Enfoque CPS: El CPS insiste en la definición de las capas y sus funciones. Un distractor común es intercambiar las funciones de la capa de negocio con la de datos. También preguntan si una arquitectura multinivel consume menos ancho de banda (la respuesta correcta es que no necesariamente).

Arquitecturas de Servicios Web (WS) y Protocolos Asociados

SOA (Service Oriented Architecture)

- **Definición:** "Modelo de arquitectura para el desarrollo de aplicaciones distribuidas basado en la utilización de servicios que, a diferencia de la orientación a objetos, son débilmente acoplados y altamente interoperables sobre estándares tecnológicos abiertos".
- **Componentes Principales:** Servicio, Proveedor de servicios, Consumidor de servicios y Registro de servicios.

Estándar SOAP (Simple Object Access Protocol)

- **Definición:** "Protocolo estándar que define cómo dos objetos en diferentes procesos pueden comunicarse por medio de intercambio de datos XML".
- **Características:** Extensibilidad (seguridad y WS-routing), Neutralidad (bajo TCP puede usarse sobre HTTP, SMTP, JMS) e Independencia.

Estándar WSDL (Web Services Description Language)

- **Definición:** "Formato XML usado para describir la interfaz pública a los servicios Web". "Permite definir la interfaz, concretamente operaciones y tipos de datos".
- **Elementos del WSDL:**
 - **Tipos de datos:** definidos en esquemas XML.
 - **Mensajes:** definen los elementos de mensaje.
 - **PortType (Tipos de puerto):** "Define las operaciones permitidas y los mensajes intercambiados".
 - **Binding:** "Especifica protocolos de comunicación usados".
 - **Service:** "Conjunto de puertos y dirección de los mismos".
- **Tipos de operaciones WSDL:**
 - **One-way (unidireccional):** recibe parámetros, no devuelve nada.
 - **Request-response (petición-respuesta):** recibe y devuelve datos.
 - **Solicit-response (solicitud-respuesta):** envía solicitud y recibe respuesta.
 - **Notification (notificación):** envía mensaje sin esperar respuesta.

Otros Estándares y Protocolos

- **UDDI (Universal Description, Discovery and Integration):** "Protocolo asociado al descubrimiento de servicios web".
- **WSIL (Web Services Inspection Language):** "Define un modo de publicar y encontrar información sobre servicios Web".

- **WS-Security (WSS):** "Estándar que define cómo debe garantizarse la integridad y seguridad en mensajería de Servicios Web".
- **WS-SecureConversation:** "Define los mecanismos para establecer y compartir contextos de seguridad".
- **SAML (Security Assertion Markup Language):** "Estándar abierto que define un esquema XML para el intercambio de datos de autenticación y autorización".

Enfoque CPS: En servicios web, CPS pregunta por la función de cada elemento XML del WSDL (especialmente binding y portType) y por el acrónimo SOA. Es frecuente encontrar preguntas sobre el protocolo de descubrimiento UDDI.

Formatos de Datos: XML (SOAP) vs. JSON (REST)

- **XML:** Usado por SOAP. Basado en etiquetas, más pesado, soporta esquemas complejos (XSD).
- **JSON:** Preferido por REST. Basado en clave-valor, mucho más ligero y fácil de leer por JavaScript.



Arquitectura REST (Representational State Transfer)

Definición y Principios

- **REST:** "Interfaz para conectar varios sistemas basados en el protocolo HTTP, sirve para obtener y generar datos y operaciones, devolviendo esos datos en formatos específicos como XML y JSON".
- **Características Principales:**

- **Sin estado (Stateless):** "El servicio no almacena información sobre el estado del cliente entre peticiones".
- **Interfaz Uniforme:** Basada en estándares como URIs, HTTP y JSON.
- **Manipulación de recursos:** Los objetos se manipulan mediante URI (Universal Resource Identifier).

Propiedades de los Métodos HTTP: Seguridad e Idempotencia

- **Idempotentes:** GET, HEAD, OPTIONS, PUT, DELETE.
- **No Idempotentes:** POST (cada vez que lo lanzas, creas un recurso nuevo).
- **Seguros:** GET, HEAD, OPTIONS (no modifican el servidor).

Métodos HTTP en REST

- **GET:** Recuperar información.
- **POST:** Crear recurso.
- **PUT:** Actualizar recurso.
- **DELETE:** Eliminar recurso.
- **OPTIONS:** "Método válido dentro del protocolo HTTP (para verificar opciones permitidas)".

Enfoque CPS: La pregunta "estrella" de REST es su naturaleza Stateless (sin estado). También suelen preguntar por el formato de datos preferido (JSON) y los códigos de estado HTTP (ej. 301 para trasladado permanentemente, 401/403 para falta de autorización, 503 para servicio no disponible).

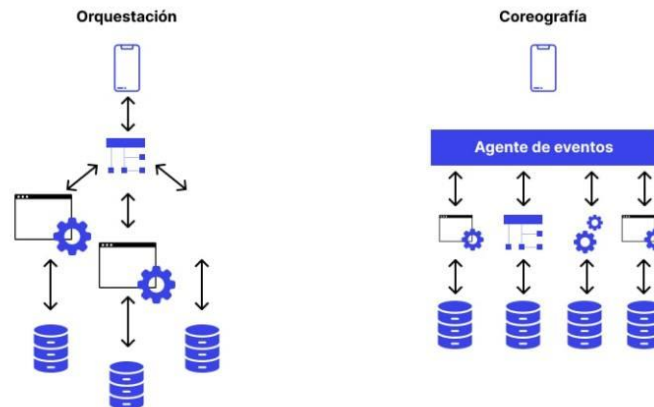
Arquitecturas Modernas: Microservicios y Contenedores

Microservicios

- **Definición:** "Arquitectura que fragmenta la aplicación en componentes menores llamados microservicios. Permiten que los procesos se ejecuten de forma aislada, administrados por diferentes equipos o lenguajes".
- **Beneficios:** "Permite escalado flexible, admite integración y entrega continuas (CI/CD), y mejora la resistencia a errores".

permitiendo degradar una funcionalidad sin bloquear toda la aplicación".

- **Gestión de composición:** Se gestiona mediante Coreografía u Orquestación.



Kubernetes y Contenedores

- **Kubernetes:** Sistema para automatizar el despliegue, escalado y gestión de aplicaciones en contenedores.
- **Objetos Básicos de Kubernetes:** Pod, Service, Volume, Namespace.
- **Docker:** "El mecanismo preferido para persistir datos en contenedores Docker son los Volúmenes".

Enfoque CPS: En microservicios, el tribunal busca que identifiques los beneficios (escalado, CI/CD) y las estrategias de gestión (orquestación). En Kubernetes, es vital memorizar los objetos básicos (Pod, Service, Volume, Namespace). Han preguntado también por la plataforma Databricks para analítica con Spark.

Modelos de Servicio Cloud: IaaS, PaaS y SaaS

- **IaaS (Infraestructura):** Alquilas el hardware (ej. AWS EC2).
- **PaaS (Plataforma):** Alquilas el entorno para desplegar (ej. Heroku, Google App Engine).
- **SaaS (Software):** Consume la aplicación final (ej. Office 365, Gmail).

Modelos de servicio cloud

